

信息安全项目实训课程

课题1：TEE 隔离环境密码接口实现

1. 课题背景和要求

在现代网络安全架构中，密码模块是信任根的重要组成部分。传统软件加解密方案中，密钥通常存在普通操作系统的内存或文件系统中。一旦系统遭受提权攻击、恶意调试或内存转储，攻击者就可能直接获取明文密钥。

课题要求：在TEE 的 **Secure World (安全世界)** 中构建一个基础的密码服务模块，将密钥的生成、保存和使用都限制在 TEE 内部。REE 侧的客户端应用（CA）只能发起命令和提交数据，不能直接读取或导出密钥。

2. 核心技术原理

- **TrustZone 硬件隔离**：将系统划分为 REE 与 TEE 两个执行域，敏感运算在 TEE 中完成。
- **TEE Internal Core API**：TA 侧使用标准内部接口完成安全对象与密码操作：
 - `TEE_AllocateTransientObject`：分配瞬时密钥对象；
 - `TEE_GenerateKey`：在 TEE 内生成密钥；
 - `TEE_AllocateOperation` / `TEE_SetOperationKey`：创建并绑定加密操作；
 - `TEE_CipherInit` / `TEE_CipherUpdate` / `TEE_CipherDoFinal`：完成分段或一次性加解密。
- **共享内存机制**：CA 与 TA 通过 `TEEC_Operation` 以及 `TEEC_MEMREF_TEMP_*` 或共享内存结构传递数据，TEE 负责边界检查。
- **会话上下文管理**：若要支持“生成密钥后在同一会话中继续使用”，可将密钥句柄或操作上下文绑定到 TA 会话上下文，而不是直接使用全局变量。

3. 详细技术流程

1. **定义通信协议**：在 `user_ta_header_defines.h` 或公共头文件中定义命令字，例如 `CMD_CREATE_KEY`、`CMD_ENCRYPT`、`CMD_DECRYPT`。
2. **TA 侧密钥管理**：TA 生成 AES 密钥并保存在会话上下文中，避免不同会话之间共享同一把实验密钥。
3. **加解密流式处理**：为了支持较大数据量，可采用 `Update` 接口分块处理；对于入门测试，也可以先用一次性 `DoFinal` 完成。
4. **CA 侧参数打包**：CA 使用 `TEEC_Operation` 配置参数类型，并通过 `TEEC_MEMREF` 传递输入输出缓冲区。
5. **结果验证**：加密后再解密，检查还原后的明文是否与原始输入一致。

4. 详细代码实现

4.1 核心头文件定义 (common.h)

```
1 #define TA_CRYPT_UUID { 0x12345678, 0xabcd, 0x1234, \  
2                               { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01 } }  
3  
4 /* 命令 ID */  
5 #define CMD_CREATE_KEY      1  
6 #define CMD_ENCRYPT          2  
7 #define CMD_DECRYPT          3  
8  
9 #define AES_KEY_SIZE_BIT    128  
10 #define AES_BLOCK_SIZE     16
```

4.2 TA 侧实现 (crypt_ta.c)

```
1 #include <tee_internal_api.h>  
2 #include <tee_internal_api_extensions.h>  
3 #include "common.h"  
4  
5 struct crypt_session {  
6     TEE_ObjectHandle key;  
7 };  
8  
9 static TEE_Result cmd_create_key(void *sess_ctx)  
10 {  
11     TEE_Result res;  
12     struct crypt_session *ctx = (struct crypt_session *)sess_ctx;  
13  
14     if (!ctx)  
15         return TEE_ERROR_BAD_PARAMETERS;  
16  
17     if (ctx->key != TEE_HANDLE_NULL) {  
18         TEE_FreeTransientObject(ctx->key);  
19         ctx->key = TEE_HANDLE_NULL;  
20     }  
21  
22     res = TEE_AllocateTransientObject(TEE_TYPE_AES, AES_KEY_SIZE_BIT, &ctx->key);  
23     if (res != TEE_SUCCESS)  
24         return res;  
25  
26     res = TEE_GenerateKey(ctx->key, AES_KEY_SIZE_BIT, NULL, 0);  
27     if (res != TEE_SUCCESS) {
```

```

28         TEE_FreeTransientObject(ctx->key);
29         ctx->key = TEE_HANDLE_NULL;
30         return res;
31     }
32
33     return TEE_SUCCESS;
34 }
35
36 static TEE_Result cmd_cipher_update(void *sess_ctx, uint32_t param_types,
37                                     TEE_Param params[4], uint32_t mode)
38 {
39     TEE_Result res;
40     struct crypt_session *ctx = (struct crypt_session *)sess_ctx;
41     TEE_OperationHandle op = TEE_HANDLE_NULL;
42     uint8_t iv[AES_BLOCK_SIZE] = {0};
43
44     uint32_t exp = TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
45                                     TEE_PARAM_TYPE_MEMREF_OUTPUT,
46                                     TEE_PARAM_TYPE_NONE,
47                                     TEE_PARAM_TYPE_NONE);
48
49     if (param_types != exp)
50         return TEE_ERROR_BAD_PARAMETERS;
51
52     if (!ctx || ctx->key == TEE_HANDLE_NULL)
53         return TEE_ERROR_NO_DATA;
54
55     if (params[0].memref.size % AES_BLOCK_SIZE != 0)
56         return TEE_ERROR_BAD_PARAMETERS;
57
58     res = TEE_AllocateOperation(&op, TEE_ALG_AES_CBC_NOPAD, mode, AES_KEY_SIZE_BIT);
59     if (res != TEE_SUCCESS)
60         return res;
61
62     res = TEE_SetOperationKey(op, ctx->key);
63     if (res != TEE_SUCCESS)
64         goto out;
65
66     TEE_CipherInit(op, iv, sizeof(iv));
67
68     /* 演示 Update 模式 */
69     res = TEE_CipherUpdate(op,
70                             params[0].memref.buffer, params[0].memref.size,

```

```

71                                     params[1].memref.buffer, &params[1].memref.size);
72
73 out:
74     if (op != TEE_HANDLE_NULL)
75         TEE_FreeOperation(op);
76     return res;
77 }
78
79 TEE_Result TA_CreateEntryPoint(void)
80 {
81     return TEE_SUCCESS;
82 }
83
84 void TA_DestroyEntryPoint(void)
85 {
86 }
87
88 TEE_Result TA_OpenSessionEntryPoint(uint32_t param_types, TEE_Param params[4],
89                                     void **sess_ctx)
90 {
91     struct crypt_session *ctx;
92
93     (void)param_types;
94     (void)params;
95
96     ctx = TEE_Malloc(sizeof(*ctx), 0);
97     if (!ctx)
98         return TEE_ERROR_OUT_OF_MEMORY;
99
100    ctx->key = TEE_HANDLE_NULL;
101    *sess_ctx = ctx;
102    return TEE_SUCCESS;
103 }
104
105 void TA_CloseSessionEntryPoint(void *sess_ctx)
106 {
107     struct crypt_session *ctx = (struct crypt_session *)sess_ctx;
108
109     if (!ctx)
110         return;
111
112     if (ctx->key != TEE_HANDLE_NULL)
113         TEE_FreeTransientObject(ctx->key);

```

```

114
115     TEE_Free(ctx);
116 }
117
118 TEE_Result TA_InvokeCommandEntryPoint(void *sess_ctx, uint32_t cmd_id,
119                                     uint32_t param_types, TEE_Param
120                                     params[4])
121 {
122     switch (cmd_id) {
123     case CMD_CREATE_KEY:
124         return cmd_create_key(sess_ctx);
125     case CMD_ENCRYPT:
126         return cmd_cipher_update(sess_ctx, param_types, params, TEE_MODE_ENCRYPT);
127     case CMD_DECRYPT:
128         return cmd_cipher_update(sess_ctx, param_types, params, TEE_MODE_DECRYPT);
129     default:
130         return TEE_ERROR_BAD_PARAMETERS;
131 }

```

4.3 CA 侧实现 (crypt_ca.c)

```

1 #include <err.h>
2 #include <stdio.h>
3 #include <string.h>
4 #include <tee_client_api.h>
5 #include "common.h"
6
7 int main(void)
8 {
9     TEEC_Result res;
10    TEEC_Context ctx;
11    TEEC_Session sess;
12    TEEC_UUID uuid = TA_CRYPT_UUID;
13    TEEC_Operation op = {0};
14    uint32_t err_origin;
15
16    /* 使用 32 字节输入, 满足 AES-CBC-NOPAD 的块对齐要求 */
17    uint8_t clear_text[32] = "Hello OP-TEE Crypto Demo";
18    uint8_t cipher_text[32] = {0};
19    uint8_t decrypted_text[32] = {0};
20

```

```

21     res = TEEC_InitializeContext(NULL, &ctx);
22     if (res != TEEC_SUCCESS)
23         errx(1, "TEEC_InitializeContext failed: 0x%x", res);
24
25     res = TEEC_OpenSession(&ctx, &sess, &uuid, TEEC_LOGIN_PUBLIC,
26                           NULL, NULL, &err_origin);
27     if (res != TEEC_SUCCESS)
28         errx(1, "TEEC_OpenSession failed: 0x%x origin=0x%x", res, err_origin);
29
30     res = TEEC_InvokeCommand(&sess, CMD_CREATE_KEY, NULL, &err_origin);
31     if (res != TEEC_SUCCESS)
32         errx(1, "CMD_CREATE_KEY failed: 0x%x origin=0x%x", res, err_origin);
33
34     op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
35                                     TEEC_MEMREF_TEMP_OUTPUT,
36                                     TEEC_NONE, TEEC_NONE);
37
38     op.params[0].tmpref.buffer = clear_text;
39     op.params[0].tmpref.size = sizeof(clear_text);
40     op.params[1].tmpref.buffer = cipher_text;
41     op.params[1].tmpref.size = sizeof(cipher_text);
42
43     res = TEEC_InvokeCommand(&sess, CMD_ENCRYPT, &op, &err_origin);
44     if (res != TEEC_SUCCESS)
45         errx(1, "CMD_ENCRYPT failed: 0x%x origin=0x%x", res, err_origin);
46
47     memset(&op, 0, sizeof(op));
48     op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
49                                     TEEC_MEMREF_TEMP_OUTPUT,
50                                     TEEC_NONE, TEEC_NONE);
51
52     op.params[0].tmpref.buffer = cipher_text;
53     op.params[0].tmpref.size = sizeof(cipher_text);
54     op.params[1].tmpref.buffer = decrypted_text;
55     op.params[1].tmpref.size = sizeof(decrypted_text);
56
57     res = TEEC_InvokeCommand(&sess, CMD_DECRYPT, &op, &err_origin);
58     if (res != TEEC_SUCCESS)
59         errx(1, "CMD_DECRYPT failed: 0x%x origin=0x%x", res, err_origin);
60
61     printf("clear      : %s\n", clear_text);
62     printf("decrypted : %s\n", decrypted_text);
63
64     TEEC_CloseSession(&sess);
65     TEEC_FinalizeContext(&ctx);

```

```
64 |         return 0;
65 |     }
```

5. 分析与讨论

- **关于全局密钥句柄**：原始写法使用全局变量保存密钥，但密钥绑定到会话上下文是更合理做法。
- **关于 Update 与 Final**：若实验只是验证“能加能解”，可直接使用 `TEE_CipherDoFinal`；若强调“分块流式处理”，则应使用 `Init + Update (+ DoFinal)` 的形式，并说明块对齐限制。
- **关于 IV**：示例代码中使用固定全 0 IV 只是为了教学简化，正式设计中必须使用随机 IV，并和密文一起保存或传输。
- **关于共享内存**：处理大块数据时，应优先使用共享内存而不是临时内存引用，以减少拷贝成本。

课题2：TEE 私钥安全存储系统

1. 课题背景与要求

在课题1中，密钥只存在于 TA 的运行内存中，一旦设备重启，密钥就会丢失。对于数字证书、设备标识、支付私钥等敏感数据，仅靠瞬时对象是不够的，必须引入持久化安全存储。

课题要求：OP-TEE 提供的安全存储可以把对象以受保护的形式落盘到 REE 文件系统，或存放到底层受保护介质中。虽然文件最终会落到普通存储设备上，但对象内容会经过机密性与完整性保护，并且与创建该对象的 TA 绑定。

2. 核心技术原理

- **Persistent Object API**：通过 `TEE_CreatePersistentObject`、`TEE_OpenPersistentObject`、`TEE_CloseAndDeletePersistentObject1` 等接口管理持久化对象。
- **Object ID**：持久化对象由一段 ID 标识，可理解为安全存储中的“逻辑文件名”。
- **对象属性提取**：对于 RSA/ECC 密钥对象，可在 TA 内提取允许导出的公钥属性，但不能把私钥明文返回给 CA。
- **REE FS 与 RPMB**：教学环境中常使用 REE 文件系统后端，实际产品中可结合 RPMB 等更强的底层安全介质。

3. 详细技术流程

1. **CA 侧发起生成请求**：向 TA 发送“生成并存储密钥”的命令。
2. **TA 侧创建瞬时密钥对象**：分配 RSA 或 ECC 容器并生成密钥对。
3. **TA 侧持久化**：将瞬时对象写入安全存储，形成持久化对象。
4. **TA 侧读取验证**：再次打开对象，只返回公钥参数给 CA。
5. **重启后验证**：系统重启后重新执行读取命令，检查对象是否仍然存在。

4. 详细代码实现

4.1 核心定义 (storage_common.h)

```
1 #define TA_STORAGE_UUID { 0x87654321, 0x4321, 0x8765, \  
2                             { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x02 } }  
3  
4 #define CMD_GENERATE_AND_STORE_RSA    1  
5 #define CMD_READ_PUBLIC_KEY          2  
6 #define CMD_DELETE_KEY                3  
7  
8 #define RSA_KEY_SIZE_BIT              2048  
9 #define MY_KEY_ID                     "USER_PRI_KEY_001"
```

4.2 TA 侧实现 (storage_ta.c)

```
1 #include <tee_internal_api.h>  
2 #include <tee_internal_api_extensions.h>  
3 #include <string.h>  
4 #include "storage_common.h"  
5  
6 static TEE_Result cmd_gen_and_store(uint32_t param_types, TEE_Param params[4])  
7 {  
8     TEE_Result res;  
9     TEE_ObjectHandle transient_key = TEE_HANDLE_NULL;  
10    TEE_ObjectHandle persistent_key = TEE_HANDLE_NULL;  
11    const char *obj_id = MY_KEY_ID;  
12    uint32_t obj_id_len = (uint32_t)strlen(obj_id);  
13  
14    (void)param_types;  
15    (void)params;  
16  
17    res = TEE_AllocateTransientObject(TEE_TYPE_RSA_KEYPAIR,  
18                                     RSA_KEY_SIZE_BIT, &transient_key);  
19    if (res != TEE_SUCCESS)  
20        return res;  
21  
22    res = TEE_GenerateKey(transient_key, RSA_KEY_SIZE_BIT, NULL, 0);  
23    if (res != TEE_SUCCESS)  
24        goto cleanup;  
25
```



```

26     res = TEE_CreatePersistentObject(TEE_STORAGE_PRIVATE,
27                                     obj_id, obj_id_len,
28                                     TEE_DATA_FLAG_ACCESS_READ |
29                                     TEE_DATA_FLAG_ACCESS_WRITE |
30                                     TEE_DATA_FLAG_OVERWRITE,
31                                     transient_key,
32                                     NULL, 0,
33                                     &persistent_key);
34     if (res == TEE_SUCCESS)
35         TEE_CloseObject(persistent_key);
36
37 cleanup:
38     if (transient_key != TEE_HANDLE_NULL)
39         TEE_FreeTransientObject(transient_key);
40     return res;
41 }
42
43 static TEE_Result cmd_read_pub_key(uint32_t param_types, TEE_Param params[4])
44 {
45     TEE_Result res;
46     TEE_ObjectHandle key_handle = TEE_HANDLE_NULL;
47     uint32_t exp = TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_OUTPUT,
48                                     TEE_PARAM_TYPE_NONE,
49                                     TEE_PARAM_TYPE_NONE,
50                                     TEE_PARAM_TYPE_NONE);
51
52     if (param_types != exp)
53         return TEE_ERROR_BAD_PARAMETERS;
54
55     res = TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,
56                                   MY_KEY_ID, strlen(MY_KEY_ID),
57                                   TEE_DATA_FLAG_ACCESS_READ,
58                                   &key_handle);
59     if (res != TEE_SUCCESS)
60         return res;
61
62     res = TEE_GetObjectBufferAttribute(key_handle, TEE_ATTR_RSA_MODULUS,
63                                       params[0].memref.buffer,
64                                       &params[0].memref.size);
65
66     TEE_CloseObject(key_handle);
67     return res;
68 }

```

```

69
70 static TEE_Result cmd_delete_key(void)
71 {
72     TEE_Result res;
73     TEE_ObjectHandle key_handle = TEE_HANDLE_NULL;
74
75     res = TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,
76                                     MY_KEY_ID, strlen(MY_KEY_ID),
77                                     TEE_DATA_FLAG_ACCESS_WRITE_META,
78                                     &key_handle);
79     if (res != TEE_SUCCESS)
80         return res;
81
82     TEE_CloseAndDeletePersistentObject1(key_handle);
83     return TEE_SUCCESS;
84 }
85
86 TEE_Result TA_InvokeCommandEntryPoint(void *sess_ctx, uint32_t cmd_id,
87                                       uint32_t param_types, TEE_Param
88   params[4])
89 {
90     (void)sess_ctx;
91
92     switch (cmd_id) {
93     case CMD_GENERATE_AND_STORE_RSA:
94         return cmd_gen_and_store(param_types, params);
95     case CMD_READ_PUBLIC_KEY:
96         return cmd_read_pub_key(param_types, params);
97     case CMD_DELETE_KEY:
98         return cmd_delete_key();
99     default:
100         return TEE_ERROR_BAD_PARAMETERS;
101 }

```

4.3 CA 侧实现 (storage_ca.c)

```

1 #include <tee_client_api.h>
2 #include <stdio.h>
3 #include <string.h>
4 #include "storage_common.h"
5

```

```

6 int main(void)
7 {
8     TEEC_Result res;
9     TEEC_Context ctx;
10    TEEC_Session sess;
11    TEEC_UUID uuid = TA_STORAGE_UUID;
12    uint32_t err_origin;
13    TEEC_Operation op = {0};
14    uint8_t pub_key_n[256] = {0};
15
16    res = TEEC_InitializeContext(NULL, &ctx);
17    if (res != TEEC_SUCCESS)
18        return 1;
19
20    res = TEEC_OpenSession(&ctx, &sess, &uuid, TEEC_LOGIN_PUBLIC,
21                          NULL, NULL, &err_origin);
22    if (res != TEEC_SUCCESS)
23        return 1;
24
25    res = TEEC_InvokeCommand(&sess, CMD_GENERATE_AND_STORE_RSA, NULL, &err_origin);
26    if (res != TEEC_SUCCESS) {
27        printf("Failed to store key: 0x%x\n", res);
28        goto out;
29    }
30
31    op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_OUTPUT,
32                                     TEEC_NONE, TEEC_NONE, TEEC_NONE);
33    op.params[0].tmpref.buffer = pub_key_n;
34    op.params[0].tmpref.size = sizeof(pub_key_n);
35
36    res = TEEC_InvokeCommand(&sess, CMD_READ_PUBLIC_KEY, &op, &err_origin);
37    if (res == TEEC_SUCCESS) {
38        printf("Public Key Modulus (first 16 bytes): ");
39        for (int i = 0; i < 16; i++)
40            printf("%02X ", pub_key_n[i]);
41        printf("...\n");
42    }
43
44 out:
45    TEEC_CloseSession(&sess);
46    TEEC_FinalizeContext(&ctx);
47    return 0;
48 }

```

课题3：TLS 技术栈集成到 TEE

1. 课题背景与要求

在 TLS 握手中，服务器需要使用私钥对握手相关数据进行签名，以证明自己持有证书对应的私钥。传统架构下，私钥通常加载在 Web 服务器进程或依赖库的地址空间中，一旦进程存在内存泄漏、越界读取或内核提权问题，私钥就有泄露风险。

课题要求：不是把完整 TLS 协议都搬进 TEE，而是实现一个**私钥签名代理**：OpenSSL 仍然负责握手状态机、证书解析与网络通信，TEE 只负责最核心的私钥签名运算。

2. 核心技术原理

- **TLS 控制面与密钥运算分离**：协议逻辑仍在 REE，私钥使用行为下沉到 TEE。
- **OpenSSL 引擎/Provider 机制**：教学上可以用回调代理的思路说明“把签名动作转发到 TEE”，但不同 OpenSSL 版本的接入方式不同。
- **TEE 非对称签名接口**：TA 侧通过 `TEE_AllocateOperation`、`TEE_SetOperationKey`、`TEE_AsymmetricSignDigest` 等接口完成签名。
- **安全收益与边界**：该实验主要保护服务器私钥，不等于“整个 TLS 会话都在 TEE 中”。

3. 详细技术流程

1. **准备私钥**：可复用实验二的安全存储机制，在 TEE 中预置一把 RSA 私钥。
2. **CA 侧签名代理**：OpenSSL 计算待签名摘要后，把摘要发给 TA。
3. **TA 侧签名处理**：TA 打开持久化私钥，执行签名并返回签名结果。
4. **集成测试**：把该代理接入测试服务端，观察 TLS 握手是否能完成。

4. 详细代码实现

4.1 核心定义 (tls_common.h)

```
1 #define TA_TLS_UUID { 0x13572468, 0x2468, 0x1357, \  
2                       { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x03 } }  
3  
4 #define CMD_TLS_SIGN_HASH 1  
5 #define RSA_KEY_ID_LABEL "TLS_SERVER_KEY"
```



```

39                                     params[1].memref.buffer,
40                                     &params[1].memref.size);
41
42 out:
43     if (op != TEE_HANDLE_NULL)
44         TEE_FreeOperation(op);
45     if (key_handle != TEE_HANDLE_NULL)
46         TEE_CloseObject(key_handle);
47     return res;
48 }
49
50 TEE_Result TA_InvokeCommandEntryPoint(void *sess_ctx, uint32_t cmd_id,
51                                     uint32_t param_types, TEE_Param
52     params[4])
53 {
54     (void)sess_ctx;
55
56     if (cmd_id == CMD_TLS_SIGN_HASH)
57         return cmd_tls_sign(param_types, params);
58
59     return TEE_ERROR_BAD_PARAMETERS;
60 }

```

4.3 CA 侧 (OpenSSL 代理) 示意实现 (tls_engine.c)

```

1 #include <stdio.h>
2 #include <tee_client_api.h>
3 #include <openssl/rsa.h>
4 #include "tls_common.h"
5
6 int tee_rsa_priv_sign_proxy(const unsigned char *digest, unsigned int digest_len,
7                             unsigned char *sigret, unsigned int *siglen)
8 {
9     TEEC_Context ctx;
10    TEEC_Session sess;
11    TEEC_Operation op = {0};
12    TEEC_UUID uuid = TA_TLS_UUID;
13    uint32_t err_origin;
14    TEEC_Result res;
15
16    res = TEEC_InitializeContext(NULL, &ctx);
17    if (res != TEEC_SUCCESS)

```

```

18         return 0;
19
20     res = TEEC_OpenSession(&ctx, &sess, &uuid, TEEC_LOGIN_PUBLIC,
21                           NULL, NULL, &err_origin);
22     if (res != TEEC_SUCCESS) {
23         TEEC_FinalizeContext(&ctx);
24         return 0;
25     }
26
27     op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
28                                     TEEC_MEMREF_TEMP_OUTPUT,
29                                     TEEC_NONE, TEEC_NONE);
30
31     op.params[0].tmpref.buffer = (void *)digest;
32     op.params[0].tmpref.size = digest_len;
33     op.params[1].tmpref.buffer = sigret;
34     op.params[1].tmpref.size = RSA_SIG_MAX_SIZE;
35
36     res = TEEC_InvokeCommand(&sess, CMD_TLS_SIGN_HASH, &op, &err_origin);
37     if (res == TEEC_SUCCESS)
38         *siglen = (unsigned int)op.params[1].tmpref.size;
39
40     TEEC_CloseSession(&sess);
41     TEEC_FinalizeContext(&ctx);
42     return (res == TEEC_SUCCESS) ? 1 : 0;
43 }

```

5. 分析与讨论

- **教学边界：**本实验保护的是“服务器私钥不落入 REE 进程地址空间”，而不是“整个 TLS 数据面都由 TEE 保护”。
- **关于 RSA_meth_set_priv_enc：**代码直接写死到某个接口，但更合理是将签名代理接到 OpenSSL 的私钥签名路径上。
- **关于性能：**引入 TEE 后，握手阶段会增加 REE/TEE 切换成本，因此实验中应提醒学生，这是一种“以私钥安全换取一定延迟开销”的方案。

课题4：TEE 隔离环境人脸与视频的加密存储

1. 课题背景与要求

视频文件、人脸图片及特征数据通常体积大、敏感度高，直接存放在普通文件系统中存在泄露与篡改风险。另一方面，TEE 的安全内存容量有限，不可能把整段视频全部载入后再统一加密。

课题要求：在 TEE 中处理大文件时，应采用分块读取、分块加密、分块写回的流式方案。

2. 核心技术原理

- **分块处理 (Chunking)**：把大文件切成固定块，每次只把一块数据送入 TEE。
- **流式加密接口**：使用 `TEE_CipherInit`、`TEE_CipherUpdate`、`TEE_CipherDoFinal` 管理连续的数据流。
- **IV/Nonce 管理**：示例中可以演示由 CA 传入 IV，但应明确说明正式实现中 IV 必须随机且唯一。
- **模式选择**：
 - 若重点演示接口调用，可采用 CBC；
 - 若强调随机访问、快进和块独立处理，则 CTR 更适合教学说明。

3. 详细技术流程

1. **初始化**：CA 打开输入文件与输出文件，并准备 IV。
2. **密钥准备**：TA 加载或生成用于媒体加密的 AES 密钥。
3. **循环迭代 (分块)**：
 - CA 读取一个固定大小的数据块；
 - CA 通过 `TEEC_Operation` 把该块交给 TA；
 - TA 调用 `TEE_CipherUpdate` 输出对应密文；
 - CA 把密文写回输出文件。
4. **收尾**：对最后一块调用 `DoFinal` 或在对齐场景下结束操作，释放句柄与文件资源。

4. 详细代码实现

4.1 核心定义 (media_common.h)

```
1 #define TA_MEDIA_UUID { 0x24681357, 0x1357, 0x2468, \  
2                             { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x04 } }  
3  
4 #define CMD_INIT_CIPHER      1  
5 #define CMD_PROCESS_CHUNK    2  
6 #define CMD_FINAL_CIPHER     3  
7  
8 #define CHUNK_SIZE           4096  
9 #define MEDIA_KEY_SIZE_BIT 128
```


4.2 TA 侧实现 (media_ta.c)

```

1 #include <tee_internal_api.h>
2 #include <tee_internal_api_extensions.h>
3 #include "media_common.h"
4
5 static TEE_OperationHandle media_op = TEE_HANDLE_NULL;
6 static TEE_ObjectHandle media_key = TEE_HANDLE_NULL;
7
8 static TEE_Result cmd_init_cipher(uint32_t param_types, TEE_Param params[4])
9 {
10     TEE_Result res;
11     uint32_t exp = TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
12                                     TEE_PARAM_TYPE_NONE,
13                                     TEE_PARAM_TYPE_NONE,
14                                     TEE_PARAM_TYPE_NONE);
15
16     if (param_types != exp || params[0].memref.size != MEDIA_BLOCK_SIZE)
17         return TEE_ERROR_BAD_PARAMETERS;
18
19     if (media_key != TEE_HANDLE_NULL)
20         TEE_FreeTransientObject(media_key);
21     if (media_op != TEE_HANDLE_NULL)
22         TEE_FreeOperation(media_op);
23
24     res = TEE_AllocateTransientObject(TEE_TYPE_AES, MEDIA_KEY_SIZE_BIT, &media_key);
25     if (res != TEE_SUCCESS)
26         return res;
27
28     res = TEE_GenerateKey(media_key, MEDIA_KEY_SIZE_BIT, NULL, 0);
29     if (res != TEE_SUCCESS)
30         return res;
31
32     res = TEE_AllocateOperation(&media_op, TEE_ALG_AES_CBC_NOPAD,
33                                 TEE_MODE_ENCRYPT, MEDIA_KEY_SIZE_BIT);
34     if (res != TEE_SUCCESS)
35         return res;
36
37     res = TEE_SetOperationKey(media_op, media_key);
38     if (res != TEE_SUCCESS)

```



```

81     (void) sess_ctx;
82
83     switch (cmd_id) {
84     case CMD_INIT_CIPHER:
85         return cmd_init_cipher(param_types, params);
86     case CMD_PROCESS_CHUNK:
87         return cmd_process_chunk(param_types, params);
88     case CMD_FINAL_CIPHER:
89         return cmd_final_cipher();
90     default:
91         return TEE_ERROR_BAD_PARAMETERS;
92     }
93 }

```

4.3 CA 侧实现 (media_ca.c)

```

1  #include <stdio.h>
2  #include <stdint.h>
3  #include <string.h>
4  #include <tee_client_api.h>
5  #include "media_common.h"
6
7  int encrypt_video(const char *in_path, const char *out_path, TEEC_Session *sess)
8  {
9      FILE *in_f = fopen(in_path, "rb");
10     FILE *out_f = fopen(out_path, "wb");
11     uint8_t buffer[CHUNK_SIZE];
12     uint8_t cipher_buffer[CHUNK_SIZE];
13     uint8_t iv[MEDIA_BLOCK_SIZE] = {0x01, 0x02};
14     TEEC_Operation op = {0};
15     uint32_t err_origin;
16     size_t read_bytes;
17
18     if (!in_f || !out_f)
19         return -1;
20
21     op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
22                                     TEEC_NONE, TEEC_NONE, TEEC_NONE);
23     op.params[0].tmpref.buffer = iv;
24     op.params[0].tmpref.size = sizeof(iv);
25     if (TEEC_InvokeCommand(sess, CMD_INIT_CIPHER, &op, &err_origin) != TEEC_SUCCESS)
26         return -1;

```

```
27
28 while ((read_bytes = fread(buffer, 1, CHUNK_SIZE, in_f)) > 0) {
29     if (read_bytes % MEDIA_BLOCK_SIZE != 0) {
30         /* 教学简化：只处理块对齐文件，非对齐情况应补齐或改为支持 Final */
31         break;
32     }
33
34     memset(&op, 0, sizeof(op));
35     op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
36                                     TEEC_MEMREF_TEMP_OUTPUT,
37                                     TEEC_NONE, TEEC_NONE);
38     op.params[0].tmpref.buffer = buffer;
39     op.params[0].tmpref.size = read_bytes;
40     op.params[1].tmpref.buffer = cipher_buffer;
41     op.params[1].tmpref.size = sizeof(cipher_buffer);
42
43     if (TEEC_InvokeCommand(sess, CMD_PROCESS_CHUNK, &op, &err_origin) != TEEC_SUCCESS)
44         break;
45
46     fwrite(cipher_buffer, 1, op.params[1].tmpref.size, out_f);
47 }
48
49 TEEC_InvokeCommand(sess, CMD_FINAL_CIPHER, NULL, &err_origin);
50
51 fclose(in_f);
52 fclose(out_f);
53 return 0;
54 }
```

5. 分析与讨论

课题5：基于 TEE 的隐私保护人脸识别系统

1. 课题背景与要求

人脸识别通常分为人脸检测、特征提取和特征比对三个阶段。真正高度敏感的是“注册模板”和“比对逻辑”，因为一旦模板泄露，用户无法像修改密码一样重新生成新的脸部特征。

课题要求：不是把整个人脸识别深度模型都放入 TEE，而是把**模板存储与比对决策**下沉到 TEE 中。REE 侧可以负责采集图像和提取特征，TEE 只负责模板保护和最终判断。

2. 核心技术原理

- **模板保护**：注册得到的人脸特征向量保存在 TEE 安全存储中。
- **向量相似度计算**：可使用余弦相似度或欧氏距离完成比对。
- **定点数优化**：若设备对浮点支持弱，可把特征做定点化处理。
- **最小输出原则**：TA 最好只返回“是否通过”或有限状态，不直接返回完整分数，避免被外部反复试探。

3. 详细技术流程

1. **注册阶段 (CA+TA) :**
 - CA 提取人脸特征向量；
 - CA 把特征向量发送给 TA；
 - TA 将其写入安全存储作为注册模板。
2. **识别阶段 (CA+TA) :**
 - CA 提取待验证特征向量；
 - CA 把向量送入 TA；
 - TA 读取已注册模板并在安全内存中完成比对。
3. **判定阶段:**
 - TA 根据阈值返回是否匹配；
 - CA 根据结果决定是否放行。

4. 详细代码实现

4.1 核心定义 (face_common.h)

```
1 #define TA_FACE_UUID { 0x35714682, 0x4682, 0x3571, \  
2                       { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x05 } }  
3  
4 #define CMD_REGISTER_FACE      1  
5 #define CMD_VERIFY_FACE       2  
6  
7 #define FEATURE_DIM            128  
8 #define MATCH_THRESHOLD_X1000 800    /* 0.8 的整数化表达 */  
9 #define FACE_DB_ID             "FACE_DB_01"
```

4.2 TA 侧实现 (face_ta.c)

[illegible]

```

44                                     TEE_DATA_FLAG_ACCESS_READ |
45                                     TEE_DATA_FLAG_OVERWRITE,
46                                     TEE_HANDLE_NULL,
47                                     params[0].memref.buffer,
48                                     params[0].memref.size,
49                                     &obj);
50     if (res == TEE_SUCCESS)
51         TEE_CloseObject(obj);
52
53     return res;
54 }
55
56 static int32_t dot_product_q15(const int16_t *a, const int16_t *b)
57 {
58     int i;
59     int32_t sum = 0;
60
61     for (i = 0; i < FEATURE_DIM; i++)
62         sum += a[i] * b[i];
63
64     return sum;
65 }
66
67 static TEE_Result cmd_verify_face(uint32_t param_types, TEE_Param params[4])
68 {
69     TEE_Result res;
70     int16_t registered_v[FEATURE_DIM];
71     int16_t *input_v = (int16_t *)params[0].memref.buffer;
72     int32_t score;
73
74     uint32_t exp = TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
75                                     TEE_PARAM_TYPE_VALUE_OUTPUT,
76                                     TEE_PARAM_TYPE_NONE,
77                                     TEE_PARAM_TYPE_NONE);
78
79     if (param_types != exp ||
80         params[0].memref.size != FEATURE_DIM * sizeof(int16_t))
81         return TEE_ERROR_BAD_PARAMETERS;
82
83     res = load_registered_template(registered_v);
84     if (res != TEE_SUCCESS)
85         return res;
86

```



```
16 op.params[0].tmpref.buffer = current_features.data();
17 op.params[0].tmpref.size = FEATURE_DIM * sizeof(int16_t);
18
19 if (TEEC_InvokeCommand(sess, CMD_VERIFY_FACE, &op, &err_origin) != TEEC_SUCCESS) {
20     std::printf("TEE verify failed\n");
21     return;
22 }
23
24 if (op.params[1].value.a == 1)
25     std::printf("Verification Passed\n");
26 else
27     std::printf("Verification Failed\n");
28 }
```

5. 分析与讨论

- **不要把“深度模型推理”与“模板保护”混为一谈：**本实验的教学重点是模板保护和安全比对，不一定要要求把整套人脸识别网络放进 TEE。
- **注册逻辑必须给出：**代码定义了 `CMD_REGISTER_FACE` 但实验省略了，实际需要补齐。

课题6：基于 TEE 的 FIDO2 认证模拟

1. 课题背景与要求

FIDO2 的核心思想是“私钥永不出安全硬件，公钥在注册时交给服务端”。在认证时，服务端下发一个随机挑战值，认证器使用私钥对挑战数据签名，服务端使用已注册公钥验签。

课题要求：TEE 扮演一个简化版“安全认证器”的角色，**注册时生成密钥对并保存私钥，认证时只开放受控签名接口。**

2. 核心技术原理

- **ECDSA P-256：**常见的 FIDO2 非对称签名算法之一。
- **挑战-响应机制：**保证每次认证都是新鲜请求，防止重放。
- **RP ID / Origin 绑定：**真实 FIDO2 会把站点标识纳入签名数据中，防止钓鱼。
- **用户验证联动：**可与实验五的人脸识别结果联动，实现“先验证用户，再允许签名”。

3. 详细技术流程

1. 注册流程 (MakeCredential) :

- CA 发起注册;
- TA 生成 ECDSA 密钥对;
- TA 保存私钥并返回公钥。

2. 认证流程 (GetAssertion) :

- 服务端下发 Challenge;
- CA 把 Challenge 传给 TA;
- TA 使用私钥签名并返回签名结果。

3. 验证阶段:

- 服务端用注册公钥验签;
- 若验签通过, 则认证成功。

4. 详细代码实现

4.1 核心定义 (fido_common.h)

```
1 #define TA_FIDO_UUID { 0x46823571, 0x3571, 0x4682, \  
2                               { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x06 } }  
3  
4 #define CMD_FIDO_REGISTER      1  
5 #define CMD_FIDO_AUTHENTICATE  2  
6  
7 #define CHALLENGE_SIZE        32  
8 #define MAX_SIGNATURE_SIZE    72    /* ECDSA DER 签名常见长度上限 */  
9 #define FIDO_KEY_ID            "FIDO_CRED_001"
```

4.2 TA 侧实现 (fido_ta.c)

```
1 #include <tee_internal_api.h>  
2 #include <tee_internal_api_extensions.h>  
3 #include <string.h>  
4 #include "fido_common.h"  
5  
6 struct ecc_public_key {  
7     uint8_t x[32];  
8     uint8_t y[32];  
9 };  
10  
11 static TEE_Result cmd_fido_register(uint32_t param_types, TEE_Param params[4])
```

```

12 {
13     TEE_Result res;
14     TEE_ObjectHandle keypair = TEE_HANDLE_NULL;
15     TEE_ObjectHandle persistent_key = TEE_HANDLE_NULL;
16     struct ecc_public_key *pub;
17     uint32_t exp = TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_OUTPUT,
18                                     TEE_PARAM_TYPE_NONE,
19                                     TEE_PARAM_TYPE_NONE,
20                                     TEE_PARAM_TYPE_NONE);
21     size_t x_len = 32, y_len = 32;
22
23     if (param_types != exp || params[0].memref.size < sizeof(struct ecc_public_key))
24         return TEE_ERROR_BAD_PARAMETERS;
25
26     pub = (struct ecc_public_key *)params[0].memref.buffer;
27
28     res = TEE_AllocateTransientObject(TEE_TYPE_ECDSA_KEYPAIR, 256, &keypair);
29     if (res != TEE_SUCCESS)
30         return res;
31
32     res = TEE_GenerateKey(keypair, 256, NULL, 0);
33     if (res != TEE_SUCCESS)
34         goto out;
35
36     res = TEE_CreatePersistentObject(TEE_STORAGE_PRIVATE,
37                                     FIDO_KEY_ID, strlen(FIDO_KEY_ID),
38                                     TEE_DATA_FLAG_ACCESS_READ |
39                                     TEE_DATA_FLAG_ACCESS_WRITE |
40                                     TEE_DATA_FLAG_OVERWRITE,
41                                     keypair, NULL, 0, &persistent_key);
42     if (res != TEE_SUCCESS)
43         goto out;
44
45     res = TEE_GetObjectBufferAttribute(keypair, TEE_ATTR_ECC_PUBLIC_VALUE_X,
46                                       pub->x, &x_len);
47     if (res != TEE_SUCCESS)
48         goto out;
49
50     res = TEE_GetObjectBufferAttribute(keypair, TEE_ATTR_ECC_PUBLIC_VALUE_Y,
51                                       pub->y, &y_len);
52     if (res != TEE_SUCCESS)
53         goto out;
54

```

[illegible]

```

98
99 out:
100     if (op != TEE_HANDLE_NULL)
101         TEE_FreeOperation(op);
102     if (key_handle != TEE_HANDLE_NULL)
103         TEE_CloseObject(key_handle);
104     return res;
105 }
106
107 TEE_Result TA_InvokeCommandEntryPoint(void *sess_ctx, uint32_t cmd_id,
108                                     uint32_t param_types, TEE_Param
109                                     params[4])
110 {
111     (void)sess_ctx;
112
113     switch (cmd_id) {
114     case CMD_FIDO_REGISTER:
115         return cmd_fido_register(param_types, params);
116     case CMD_FIDO_AUTHENTICATE:
117         return cmd_fido_authenticate(param_types, params);
118     default:
119         return TEE_ERROR_BAD_PARAMETERS;
120 }

```

4.3 CA 侧实现 (fido_ca.c)

```

1 #include <stdio.h>
2 #include <stdint.h>
3 #include <tee_client_api.h>
4 #include "fido_common.h"
5
6 struct ecc_public_key {
7     uint8_t x[32];
8     uint8_t y[32];
9 };
10
11 int main(void)
12 {
13     TEEC_Context ctx;
14     TEEC_Session sess;
15     TEEC_Operation op = {0};

```

```

16     TEEC_UUID uuid = TA_FIDO_UUID;
17     struct ecc_public_key pub_key;
18     uint8_t challenge[CHALLENGE_SIZE] = {0xAA, 0xBB};
19     uint8_t signature[MAX_SIGNATURE_SIZE];
20     uint32_t err_origin;
21     TEEC_Result res;
22
23     res = TEEC_InitializeContext(NULL, &ctx);
24     if (res != TEEC_SUCCESS)
25         return 1;
26
27     res = TEEC_OpenSession(&ctx, &sess, &uuid, TEEC_LOGIN_PUBLIC,
28                           NULL, NULL, &err_origin);
29     if (res != TEEC_SUCCESS)
30         return 1;
31
32     op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_OUTPUT,
33                                     TEEC_NONE, TEEC_NONE, TEEC_NONE);
34     op.params[0].tmpref.buffer = &pub_key;
35     op.params[0].tmpref.size = sizeof(pub_key);
36     res = TEEC_InvokeCommand(&sess, CMD_FIDO_REGISTER, &op, &err_origin);
37     if (res != TEEC_SUCCESS) {
38         printf("FIDO register failed\n");
39         goto out;
40     }
41
42     printf("FIDO register success\n");
43
44     memset(&op, 0, sizeof(op));
45     op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
46                                     TEEC_MEMREF_TEMP_OUTPUT,
47                                     TEEC_NONE, TEEC_NONE);
48     op.params[0].tmpref.buffer = challenge;
49     op.params[0].tmpref.size = sizeof(challenge);
50     op.params[1].tmpref.buffer = signature;
51     op.params[1].tmpref.size = sizeof(signature);
52
53     res = TEEC_InvokeCommand(&sess, CMD_FIDO_AUTHENTICATE, &op, &err_origin);
54     if (res != TEEC_SUCCESS) {
55         printf("FIDO authenticate failed\n");
56         goto out;
57     }
58

```

```
59     printf("FIDO authenticate success, signature size = %zu\n",
60            op.params[1].tmpref.size);
61
62 out:
63     TEEC_CloseSession(&sess);
64     TEEC_FinalizeContext(&ctx);
65     return 0;
66 }
```

5. 分析与讨论

- **FIDO2 不只是“签一个随机数”**：真实协议还会把 RP ID、客户端数据哈希、计数器等纳入认证材料。实训文档应明确说明这里是“简化模拟版”。
- **和实验五的衔接**：把用户存在性验证接到签名前，是很好的扩展方向。
- **关于签名长度**：ECDSA 签名通常是 DER 编码，不宜简单按 128 字节固定长度去讲解。